Research Plan

Consolidated from: `paper/QuantumWeekOverleaf/ideas.txt`, `docs/parked_ideas.md`, `docs/experimental_insights.md`, `docs/evaluation_diagnosis.md`, `docs/iterative_expansion_report.md`, `pattern_sequences.txt`, `Quantum Algorithm Composition Hierarchie.txt`.

---

1. Done

1.1 Run analysis and update paper numbers

Re-run with the identifier normalizer active and update tables in the paper. Completed. Final run results are in `data/runs/`.

1.2 Identifier normalization

Split `snake_case` / `CamelCase` identifiers into space-separated tokens before embedding. Improved total matches from 751 → 805 (+7.2%) and name-based matches from 475 → 531 (+11.8%). Multi-word patterns (Grover, VQA, Amplitude Amplification) gained the most.

1.3 KB labelling errors fixed

- `EstimationProblem` relabelled from Oracle → Quantum Amplitude Estimation.
- `AmplitudeEstimation` family relabelled from Amplitude Amplification → Quantum Amplitude Estimation.
- Impact: qiskit-finance Micro F1 jumped from 0.000 → 0.300.

1.4 Verbose CSV cleanup

Removed the `patterns_in_kw_comment:*`, `patterns_in_caller:*`, and similar channel-based CSV files. Added a `:` guard in `csv_exporter.py` and `generate_report.py` so they are never generated.

1.5 Qrisp execution tracer (experiment)

`experiments/qrisp_execution_trace.py`: downloads 5 Qrisp tutorial notebooks, executes them with `uv run --with qrisp`, traces all Qrisp API calls via `sys.settrace`, filters to quantum-related calls, and prints the execution sequence per notebook. Results written to `experiments/results/qrisp_trace/`. Run with `just trace-qrisp`.

1.6 Inter-rater agreement study

Full Fleiss'/Light's Kappa report for Qiskit Algorithms, Qiskit ML, and Qrisp GT sheets. Report at `data/report/kappa_interrater_agreement.txt`. - Qiskit Algorithms: Light's κ=0.717, Fleiss' κ=0.696 - Qiskit ML: Light's κ=0.576, Fleiss' κ=0.566 - Qrisp GT (per-label binary): Light's κ=0.345

---

2. Implemented and Reverted — Re-enable After KB Expansion

2.1 Cell-level comment matching

Instead of embedding the whole-file comment blob, extract the Jupyter cell markdown description (lines between `# In[N]:` boundaries) and associate it with the call sites in that code cell.

Reverted because it added false positives without improving recall: the bottleneck is KB vocabulary gap, not comment granularity. A cell saying "Bernstein-Vazirani finds a hidden string" still can't match because the KB has no Oracle concept using that vocabulary.

Re-enable when: the KB has Zoo/wildcard entries covering missing patterns. At that point, cell-level context would be strictly better than whole-file averaging.

Implementation: cell-boundary parser (`# In[N]:` markers) was written and validated. Swap `extract_comments_from_script` back to the cell-boundary version in `run_analysis.py`.

---

3. Designed, Not Yet Implemented

3.1 Quantum Zoo KB extension (highest expected impact for Oracle pattern)

Parse the Quantum Algorithm Zoo (`stephenjordan/stephenjordan.github.io/blob/master/index.html`) and add each algorithm's description as a wildcard KB entry mapped to the appropriate Pattern Atlas pattern.

Design decisions already made: - Map by "what the algorithm IS", not "what subroutines it uses". - Algorithms that are canonical examples of two patterns get two KB entries (different concept names, same summary). - ~30 unambiguous mappings; ~6 edge cases. - Zoo categories map to: Oracle/Grover (Oracular), QPE/QFT (Algebraic), QAOA/VQA (Optimization). Simulation category has no matching pattern — leave unmapped.

Expected impact: - Oracle pattern recall: ~0.000 → ~0.6+ (BV, DJ, Simon's descriptions match Oracle language). - Domain Specific Application recall: improvement for Shor's, Walk, QMC. - No QML impact (Zoo has limited ML coverage).

Next step: write a parser for the Zoo HTML → generate `data/zoo_kb_entries.json` → load alongside existing KB entries at analysis time.

3.2 Defined-class matcher (highest impact for library-level code)

The call-site matcher cannot detect what a file *implements* — it only sees what the file *calls*. For library-definition code (Qrisp, Qualtran), recall collapses to near-zero because internal composition primitives are called, not framework-level algorithm names.

Proposed fix: extract the docstrings of every class and function *defined* in the target file and embed those against pattern summaries. This complements the call-site matcher and covers library files directly.

Example: `alg_primitives/qpe.py` defines `class QuantumPhaseEstimation` with a docstring explaining QPE. Embedding that docstring should match the QPE pattern summary at high similarity, regardless of what functions the file calls internally.

Expected impact on current benchmarks: - Qrisp: F1=0.058 → substantially higher (most files have docstrings) - Qualtran: F1=0.051 → substantially higher - qiskit-finance (objective files): Oracle recall ~0 → improved

Implementation sketch: 1. After AST parsing, collect all `FunctionDef` / `ClassDef` nodes in the file. 2. Extract their docstrings (first string literal in the body). 3. Concatenate and embed as a second "description" text for the file. 4. Run summary matching against that text with its own threshold (suggested 0.70). 5. Merge results with call-site matches; deduplicate by (file, concept).

3.3 QML ground truth supplement (qiskit-machine-learning)

Amazon Braket Algorithm Library has zero QML content, so the current GT cannot measure recall for QNN, VQC, quantum kernel methods, or variational classifiers.

Best candidate: `qiskit-machine-learning` — calls functions already in the KB (`SamplerQNN`, `EstimatorQNN`, `QNNCircuit`, `VQC`, `QSVC`, `FidelityQuantumKernel`) so name matching fires reliably.

Patterns added that braket lacks: Quantum Neural Network, VQA as classifier, Data Encoding (ZZFeatureMap, PauliFeatureMap), quantum kernel methods.

Patterns still absent after braket + qiskit-ml: Quantum Error Correction, Hamiltonian Simulation as a standalone application.

Next step: annotate ~30 qiskit-machine-learning notebooks and add to GT corpus.

3.4 Per-framework observed recall as the primary metric

Standard micro-averaged F1 conflates framework specialisation with tool failure. A framework with no QML content accumulates QNN false negatives that are not tool failures — those patterns simply aren't there.

Correct metric: for each (framework, pattern) pair where the framework's GT contains ≥ 1 positive, compute TP / (TP + FN). Average only over patterns present in that framework's GT ("observed recall"). Report precision separately to flag false positives.

Next step: update the evaluation script to compute per-framework observed recall and add a table in the paper alongside the current F1 table.

3.5 Self-expanding KB via high-confidence matches

Run analysis on the reference frameworks and collect all concepts with `name_score ≥ 0.99` AND `summary_score ≥ 0.90`. Promote these as new KB entries. Re-run and measure the delta (new concepts found). Repeat until delta = 0.

A prototype was run (`docs/iterative_expansion_report.md`). It converged after 6 iterations, adding 43 new entries and growing total matches from 805 → 2151.

Open questions before using in production: - Risk of circular reinforcement: KB seeded from framework X may over-fit to X vocabulary and hurt generalization. - A held-out validation set is needed per expansion round to catch recall degradation on unseen frameworks. - Thresholds (0.99/0.90) were not tuned; should be validated against the inter-rater agreement baseline.

Status: exploratory experiment only. Do not use expanded KB in main results until the validation risk is addressed.

3.6 Remaining KB labelling fixes

Two known errors not yet fixed (documented in `docs/evaluation_diagnosis.md`):

- `Isometry` (Qiskit): current label Circuit Construction Utility → should be Data Encoding (used in `NormalDistribution` / `LogNormalDistribution` for amplitude encoding of probability distributions).

- `TrotterQRTE` (qiskit-algorithms): current label Initialization → should be Hamiltonian Simulation (Trotterized real-time evolution is simulation, not state prep).

Next step: update `enriched_qiskit_algorithms_quantum_patterns.csv` and `enriched_qiskit_quantum_patterns.csv` for these two entries, then re-run evaluation.

---

4. Data / Reference Material (No Implementation Required)

4.1 Algorithm composition sequences

`pattern_sequences.txt` lists 16 algorithms as ordered primitive sequences, e.g.: - Grover: `State Prep → Oracle → Diffuser → Measurement` - Shor: `State Prep → QPE (Mod Exp) → Inv QFT → Measurement → Classical CF` - HHL: `State Prep → QPE → Controlled Rot → Inv QPE → Measurement`

4.2 Algorithm composition hierarchies

`Quantum Algorithm Composition Hierarchie.txt` lists 15 algorithms as pattern dependency trees, e.g.: - HHL → LCU → QPE → Amplitude Amplification → Basis Change (QFT) → Initialization - GQSP → LCU → Basis Change → QPE → Hamiltonian Simulation → Quantum Arithmetic

Cross-cutting primitives: Basis Change, Amplitude Amplification, Initialization, Oracle, Quantum Arithmetic, Hamiltonian Simulation.

These two files are reference data. They could serve as the ground truth for a pattern-sequence analysis (e.g., detect whether a notebook follows an expected primitive sequence) but no such analysis is currently planned.

---

5. Early-Stage Ideas (Not Specified)

5.1 Expressiveness metric via lines of code

The fewer lines of code a framework needs for a canonical algorithm, the more expressive it is. Map a standard set of algorithms across all frameworks and compare LOC per algorithm.

No spec beyond the initial note. Would need: a fixed algorithm set, a LOC counting method (logical lines, excluding boilerplate), and a corpus of implementations per framework.

5.2 Common primitive mapping across frameworks

Identify which quantum algorithm primitives appear across all analysed frameworks and build an explicit cross-framework primitive index. The claim: there is a common set of primitives that are easier to find because they can be searched for by name. This is related to the Zoo KB extension (3.3) but broader.

No spec. The pattern_sequences and composition hierarchy files (section 4) are a starting point.